

Software Design Document (SDD)

Module: serial - Formatted Serial Output Module

Version: 1.0

Authors: Steven McClellan, Vania Leal, Leonardo Garcia, Melanie Picen

Item	Description
Source files	serial.c / serial.h
Main purpose	Provide a small formatted serial output layer on top of the UART driver and utility formatter.
Main public functions	serial_init(), serial_printf()

1. Introduction

1.1 Purpose

This document describes the design and architecture of the serial module. The module provides a lightweight formatted-output interface for embedded applications. It initializes the lower-level UART driver and formats printf-style text into a local transmit buffer before sending each character through `uart_write()`.

1.2 Scope

The serial module sits above the UART driver and the utility formatting library. It does not manipulate USART registers directly. Instead, `serial_init()` delegates initialization to `uart_init()`, while `serial_printf()` delegates string formatting to `utils_vsnprintf()` and byte transmission to `uart_write()`. The module is intended for debug/status output over USART2 or another UART instance supported by the lower-level driver.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
UART	Universal Asynchronous Receiver/Transmitter used for serial communication.
USART2	STM32 serial peripheral used by the UART driver in this project.
API	Application Programming Interface.
TX	Transmit path from microcontroller to external terminal.
va_list	C standard mechanism used to process variable arguments.
SDD	Software Design Document.

1.4 Requirements Traceability

Req. ID	Function	Requirement Statement
FR-1	<code>serial_init()</code>	The system shall initialize the serial output path by calling the lower-level UART initialization routine.
FR-2	<code>serial_printf()</code>	The system shall accept a printf-style format string and variable argument list.
FR-3	<code>serial_printf()</code>	The system shall format the message into a local buffer using <code>utils_vsnprintf()</code> .
FR-4	<code>serial_printf()</code>	The system shall transmit the formatted buffer one byte at a time using <code>uart_write()</code> .
FR-5	<code>serial_printf()</code>	The system shall terminate argument processing with <code>va_end()</code> before returning.

Non-functional requirements: no dynamic memory allocation; simple stack-based buffer; no direct USART register access inside this module; transmit behavior depends on the blocking/timeout behavior implemented by `uart_write()`.

2. System Architecture

2.1 High-Level Design

The module follows a layered architecture. The application calls the serial API. The serial layer formats messages and delegates hardware-specific work to the UART driver. The utility module provides the formatter used to expand `%d`, `%u`, `%x`, `%s`, `%c`, and `%%` specifiers, depending on the supported implementation of `utils_vsnprintf()`.

Layer	Files / Components	Description
Application Layer	main.c or higher-level application code	Calls <code>serial_init()</code> and <code>serial_printf()</code> to print debug/status messages.
Serial Layer	serial.c / serial.h	Owns the local TX buffer, handles variable arguments, and sends formatted bytes.
Utility Layer	utils.c / utils.h	Formats the message into the serial buffer through <code>utils_vsnprintf()</code> .
UART Driver Layer	uart_driver.c / uart_driver.h	Initializes USART2 and transmits one byte using <code>uart_write()</code> .
Hardware Layer	USART2 and board wiring	Physical serial peripheral and external terminal/debug console.

2.2 Class Diagram

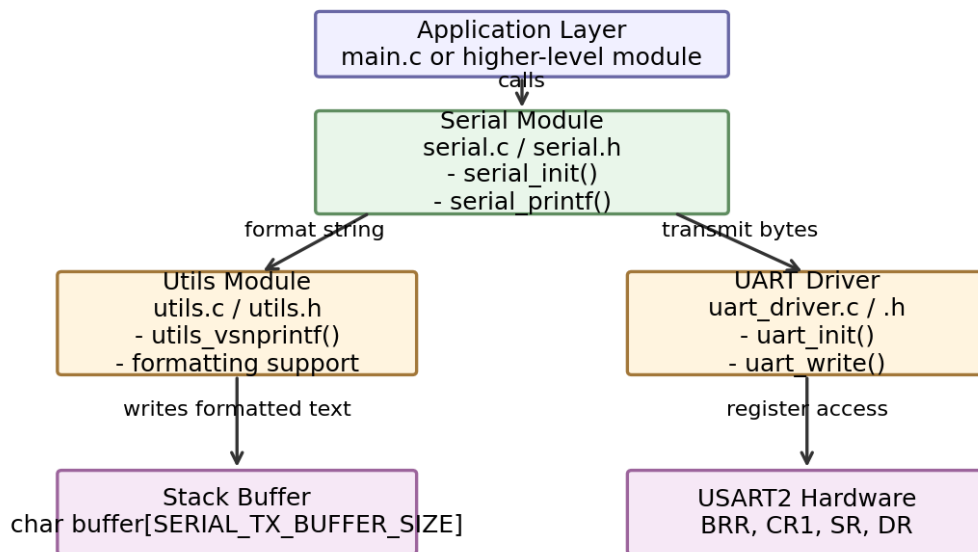


Figure 1. Class Diagram - Serial module.

2.3 Module Interfaces

Interface	Description
Public API	<code>serial_init()</code> and <code>serial_printf()</code> declared in <code>serial.h</code> .
External dependencies	<code>utils.h</code> for <code>utils_vsnprintf()</code> ; <code>uart_driver.h</code> for <code>uart_init()</code> and

	uart_write(); stdarg.h and stddef/stdint types through headers.
Buffer dependency	SERIAL_TX_BUFFER_SIZE must be defined in serial.h and large enough for the expected formatted messages.

3. Detailed Design

3.1 Module Behavior

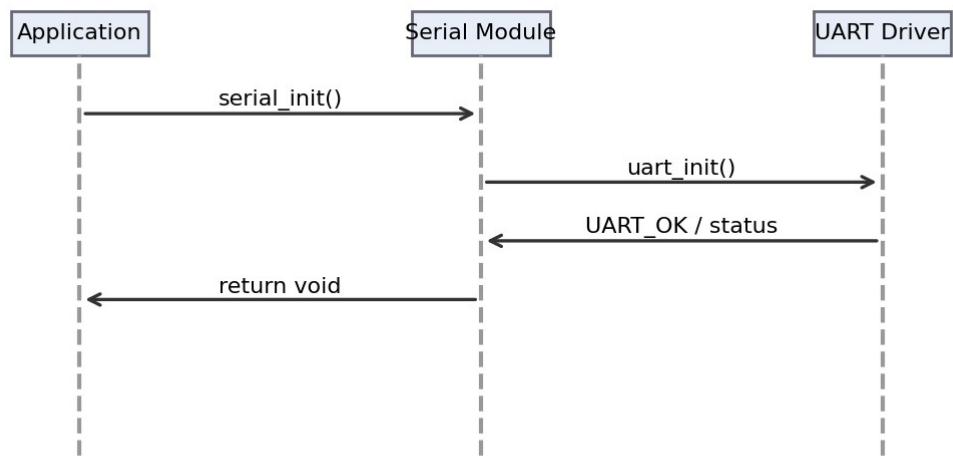


Figure 2. Sequence Diagram - serial_init.

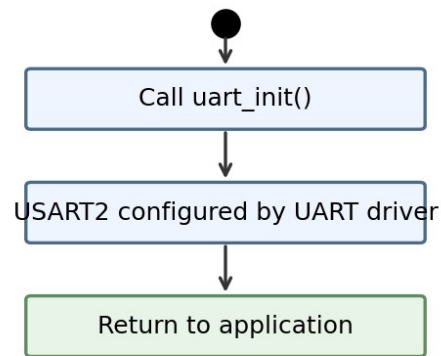


Figure 3. Activity Diagram - serial_init.

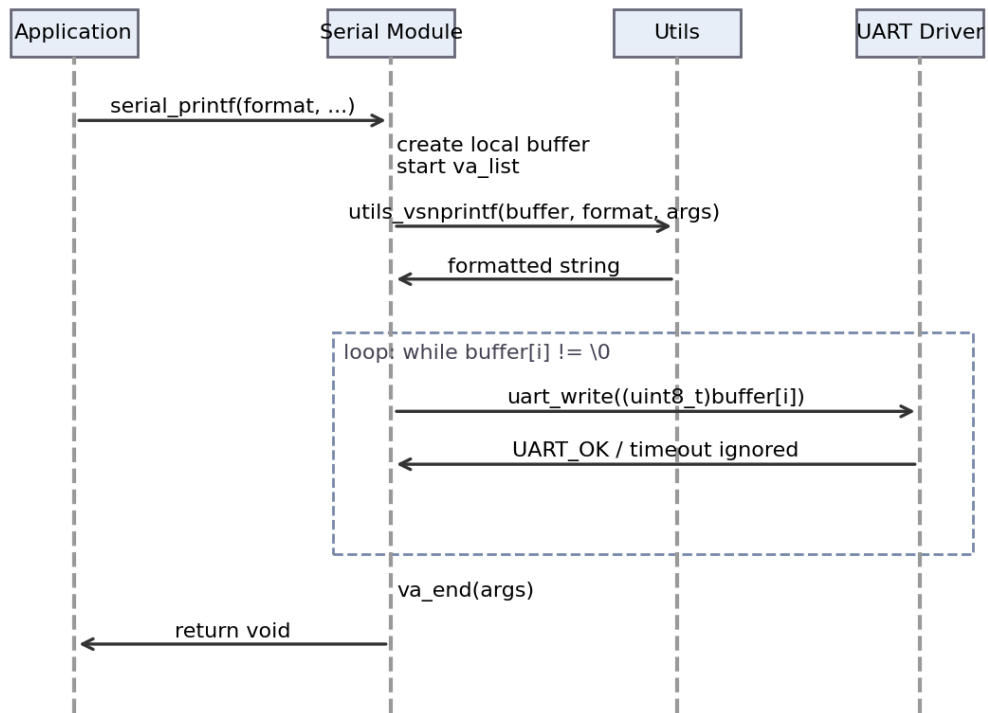


Figure 4. Sequence Diagram - `serial_printf`.

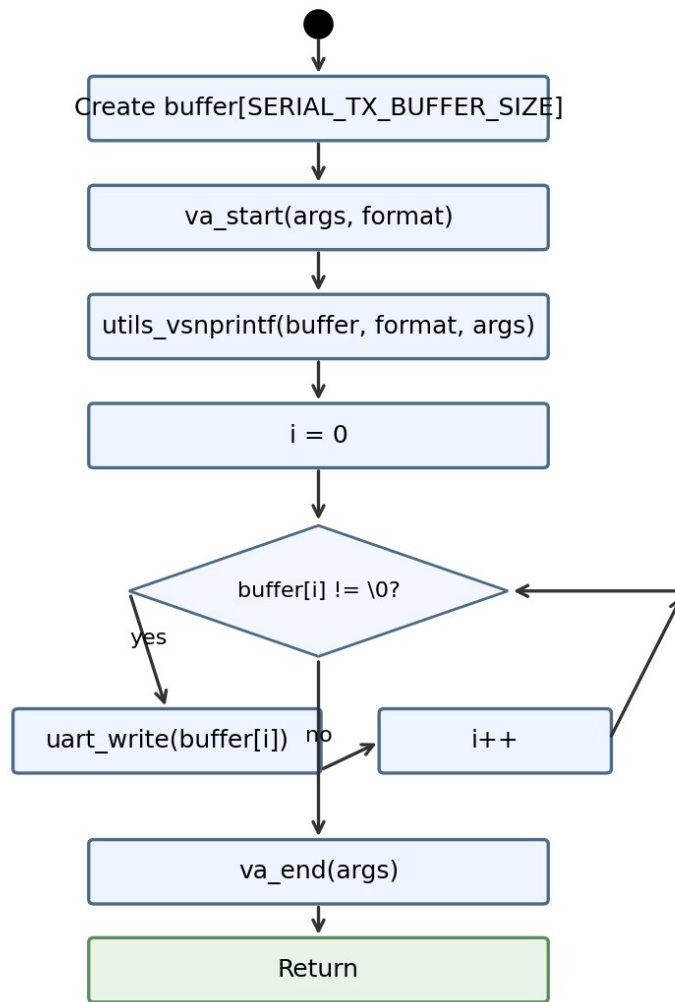


Figure 5. Activity Diagram - serial_printf.

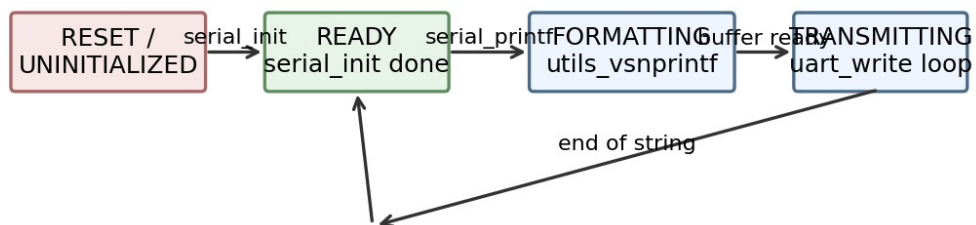


Figure 6. State Diagram - Serial module lifecycle.

3.2 serial Module

The serial module is intentionally small. `serial_init()` contains one operation: it calls `uart_init()` and returns. This keeps all UART register configuration inside the UART driver. `serial_printf()` creates a fixed-size stack buffer, starts a variable-argument list, formats the message using `utils_vsnprintf()`, and then iterates over the resulting null-terminated string. Each character is cast to `uint8_t` and sent with `uart_write()`.

3.3 Formatting Flow

The formatting responsibility is delegated to `utils_vsnprintf()`. The serial module does not parse format specifiers itself; it only supplies the destination buffer, format string, and argument list. This separation allows the formatter to be reused by other modules and keeps `serial_printf()` focused on transmission.

3.4 Transmission Flow

After formatting, the buffer is transmitted using a for loop until the null terminator is reached. The serial module does not inspect the return value of `uart_write()`; therefore, UART timeout or transmission errors are not propagated to the caller in the current implementation.

4. Application Programming Interfaces

4.1 Data Types and Macros

Name	Type / Value	Notes / Definition
<code>SERIAL_TX_BUFFER_SIZE</code>	Macro defined in <code>serial.h</code>	Size of the local transmit buffer used by <code>serial_printf()</code> .
<code>format</code>	<code>const char *</code>	Printf-style format string passed by the caller.
<code>buffer</code>	<code>char[]</code>	Local stack buffer used to store the formatted output before transmission.
<code>va_list args</code>	C standard variable-argument object	Stores the variable arguments passed to <code>serial_printf()</code> .

4.2 Error Codes

The public functions in this module have void return type and do not return error codes. Lower-level UART errors may occur inside `uart_write()`, but `serial_printf()` currently ignores that return value.

Code	Notes / Definition
Not applicable	<code>serial_init()</code> and <code>serial_printf()</code> return void in the current implementation.
<code>UART_ERR_TIMEOUT</code>	Possible lower-level <code>uart_write()</code> result, not propagated by <code>serial_printf()</code> .

4.3 Function Definitions

`serial_init`

Field	Definition
Function Name	<code>serial_init</code>
Syntax	<code>void serial_init(void)</code>
Parameters (in)	None.
Parameters (inout)	None.
Parameters (out)	None.
Return value	void.
Description	Initializes the serial output path by calling <code>uart_init()</code> .
Constraints	Does not return the <code>uart_init()</code> status to the caller.
Available via	<code>serial.h</code>

serial_printf

Field	Definition
Function Name	serial_printf
Syntax	void serial_printf(const char *format, ...)
Parameters (in)	format: printf-style format string; ...: variable arguments used by the format string.
Parameters (inout)	None.
Parameters (out)	None directly; formatted characters are transmitted through UART.
Return value	void.
Description	Formats text into a local buffer using utils_vsnprintf(), then sends each character using uart_write().
Constraints	No format pointer validation is performed in this function. Output length depends on SERIAL_TX_BUFFER_SIZE and the behavior of utils_vsnprintf(). uart_write() return value is ignored.
Available via	serial.h

4.4 Internal Function Definitions

No private helper functions are implemented inside serial.c. Formatting and byte transmission are delegated to external modules.

External Call	Purpose
uart_init()	Initializes the UART driver and USART peripheral.
utils_vsnprintf(buffer, format, args)	Builds the formatted output string.
uart_write((uint8_t)buffer[i])	Transmits one formatted character.

5. Implementation Notes and Constraints

Topic	Note
Fixed buffer	serial_printf() uses a fixed local buffer of SERIAL_TX_BUFFER_SIZE bytes. Oversized formatted strings depend on the safety behavior of utils_vsnprintf().
Blocking behavior	Transmission can block because uart_write() waits for the UART transmit-ready flag.
Error propagation	uart_write() returns a status, but serial_printf() ignores it. The application cannot detect serial timeouts through this API.
No RX path	This module provides transmit/print functionality only. It does not read UART data.
Dependency direction	serial depends on utils and uart_driver; neither the utils module nor the UART driver should depend on serial.

6. Future Enhancements

- Change serial_printf() to return a serial_status_t so UART timeout/errors can be reported to the caller.
- Add bounds-aware formatting or explicit truncation handling to prevent buffer overflow if the utility formatter does not enforce limits.
- Add serial_write_string() for simple null-terminated strings without formatting overhead.
- Add optional newline conversion from LF to CRLF for terminal compatibility.
- Add unit tests for formatting delegation, byte iteration, empty strings, and long messages.